Robot Movement Planning

AUTHORS

Andrea Pezzuolo - 381310
Andrea Agnetti - 380371

2026-01-19

Contents

1 Introduction **1**

2 Modeling the problem in Minizinc **2**
2.1 Matrix Representation 2
2.2 Bounding box representation 4
2.3 Intermediary modeling 5
2.3.1 A first attempt at symmetry breaking 6
2.4 Changing representation 7
2.5 Conveyor belt mechanism 9
2.5.1 Implementing `NULL-moves` 9
2.5.2 Implementing the Conveyor Belt 10
2.5.3 Padding Moves Symmetries 11

3 Extra **11**
3.1 Global constraint 11
3.2 Symmetries 12
3.3 Generating Redundant Constraints 14
3.4 Testing Automation 15
3.4.1 Loops 16
3.4.2 Double "T" Pattern 17

4 Results **18**

5 Conclusions **18**

List of pictures **I**

List of tables **II**

References **III**

1 Introduction

The problem of Robot movement planning admits multiple formulations and solution strategies, depending on the level of abstraction and the modeling choices adopted. In the following, we provide a formal definition of the problem addressed in this work.

Problem definition

The problem consists of placing a set of k rectangular boxes onto a pallet of size $n \times m$. All boxes share the same dimensions $x \times y$ and can be rotated by $90°$. For each box, the final destination position on the pallet is given as input, together with its required orientation. The final configuration is guaranteed to be non-overlapping.

Boxes arrive on a continuous conveyor belt, and a robotic arm is responsible for picking them up and placing them onto the pallet in their designated positions. The robotic arm is equipped with a gripper composed of two parallel plates, whose effective size along the x-axis is denoted by S. The gripper is capable of picking up multiple boxes simultaneously, provided that a number of geometric constraints are satisfied. More specifically, a set of boxes can be picked in a single action if:

- the boxes fill a contiguous space (no spaces between them in the final pallet configuration);
- let T be the sum of the lengths of the boxes being picked, then $S + x <= T$.

Once the boxes have been transported above the pallet, the gripper releases them by opening one of its plates. Due to mechanical constraints, only one plate can be opened at a time. During this operation, collisions with boxes that have already been placed on the pallet must be avoided. To increase flexibility and reduce the risk of collisions, the robotic arm is allowed to rotate the gripper by $90°$ before releasing the boxes.

Additional collisions may arise during the placement phase due to *gripper overflow*. Gripper overflow occurs when the gripper size S exceeds the length of an individual box. In such cases, parts of the gripper may extend beyond the box being placed, potentially causing collisions with neighboring boxes already present on the pallet. These constraints restrict the admissible order in which boxes can be placed.

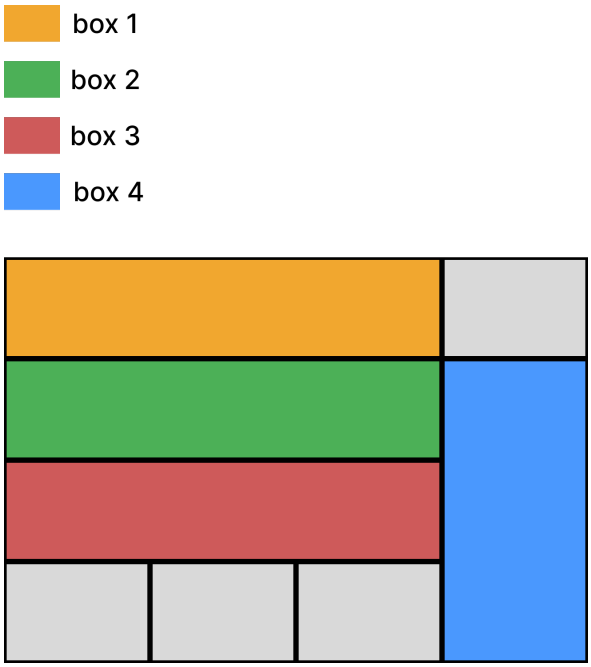


Figure 1: Example of final result

Figure 1 shows a possible solution for a simple pallet configuration on a $3 \times 3 grid$, with 4 boxes of size 2×1 and `gripper size` $= 3$. Geometrical constraints enforce that box number 4 is placed after boxes 2 and 3 due to `gripper overflow`. Box number 4 must also be placed with the gripper rotated $90°$ to the right. This configuration requires a total of 4 time steps, since no two boxes can be picked up at once.

2 Modeling the problem in Minizinc

In this section we present the chronological evolution of our solution, from the first prototype to the final working model. We present the main milestones we achieved: starting from a matrix based representation with no concept of a continuous conveyor belt, up to a bounding box representation, gradually implementing new features and improving the computational efficiency.

2.1 Matrix Representation

The first CSP used an explicit matrix representation both for the goal pallet configuration and the pallet state at each time step `t`. Each cell in the matrix was assigned a value $l \in \{0, 1\}$, where $l = 1$ meant that the cell was currently occupied by a box (a box could occupy more than one cell).

This model had few parameters since all the boxes were available from the beginning and only one box at a time could be picked up.

```
int: MAX_GRID_SIZE_X;
int: MAX_GRID_SIZE_Y;
int: GRIPPER_SIZE;
int: N_BOXES;
int: LOCAL_BOX_X_SIZE;
int: LOCAL_BOX_Y_SIZE;
array[GRID_SIZE_Y, GRID_SIZE_X] of bool: PALLET_CONFIGURATION;
```

Listing 1: Input parameters for the first model

This CSP was actually much more complex (in terms of search space) than our current model, since no information on where to drop the boxes was given to the solver. The solver had to try to fill out the entire matrix by placing boxes while still adhering to the collision related constraints.

This modeling, even though very inefficient, was helpful to define the `collision constraints` on the gripper opening.

```
predicate no_boxes_above(var TIMESTEPS: t, var GRID_SIZE_Y: start_y) =
let {
var GRID_SIZE_X: check_end_x = drop_place_x[t] + picked_item_width[t] - 1;
var GRID_SIZE_Y: check_end_y = drop_place_y[t];
} in
forall(t2 in 1..t) (
    (drop_place_y[t2] >= start_y /\ drop_place_y[t2] < check_end_y) ->
    (
        drop_place_x[t2] +  picked_item_width[t2]  < drop_place_x[t]
        \/
        drop_place_x[t2] > check_end_x
    )
);
```

Listing 2: Collision constraint checking that opening the gripper with rotation `TOP` doesn't cause any collision with already-placed boxes.

As shown in 2 collisions on boxes are checked by finding a separating axis between the two bounding boxes. In the case of `no_boxes_above`, boxes that fall within the gripper opening range (equal to 1) must be separated by a vertical line (i.e. one box must lie to the left/right of the other one). The same procedure is applied to the other possible rotations such as `RIGHT`, where a horizontal line must exist instead.

This kind of constraint is also used to check for `gripper overflow` collisions. In the case of rotation = `TOP`, collisions both on the left and the right of the just-placed box must be checked. This constraint will be kept in all future modelings since it is the standard way of checking collisions between axis-aligned bounding boxes.

The pallet state is updated at each `timestep` by filling out with 1's the area covered by the just-placed box and by bringing forward the rest of the state by inertia. This constraint is shown in Code Snippet 3. An additional constraint ensures that the final pallet state corresponds to the input one.

```
constraint forall(t in TIMESTEPS)(
  forall(i in GRID_SIZE_Y, j in GRID_SIZE_X)(
```

```
    let {
      var bool: is_in_drop_zone =
        % (i,j) falls within the bounding box of the box placed at time t
    } in
    if is_in_drop_zone then
      pallet_state[t,i,j] = 1
    else
      % inertia
      pallet_state[t,i,j] = pallet_state[t-1,i,j]
    endif
  )
);
```

Listing 3: Pallet state framing and update constraint.

This model with the matrix was very computationally inefficient and slow during testing. Because of this we decided to slightly change the approach to the problem.

2.2 Bounding box representation

The next modeling requires simplifying the CSP by adding one crucial assumption: the final goal is not defined as a matrix of 1's and 0's but as an array of boxes with their corresponding final positions. The solver is not required to compute the drop locations for the boxes, only the timing of the drops. This will also allow for a simple but effective way of handling multiple boxes being picked up at the same time, as we will see in Section 2.5.3.

The goal configuration is defined as a "structure of arrays" where each index corresponds to a box. Every box has 5 fields: its orientation (horizontal or vertical) and the 4 points used to describe the bounding box (AABB).

```
array[BOXES] of Orientation: box_orientation = array1d(BOXES, [H,V]);
% bounding boxes
array[BOXES] of Grid_size_x: box_bbox_min_x = array1d(BOXES, [1,1]);
array[BOXES] of Grid_size_y: box_bbox_min_y = array1d(BOXES, [1,2]);
array[BOXES] of Grid_size_x: box_bbox_max_x = array1d(BOXES, [3,1]);
array[BOXES] of Grid_size_y: box_bbox_max_y = array1d(BOXES, [1,4]);
```

Listing 4: Input structure of arrays.

Code Snippet 4 defines a pallet with two boxes: the first one with horizontal orientation and with its corresponding `AABB` being `[(1,1), (3,1)]`, while the second has vertical orientation and bounding box `[(1,2), (1,4)]`.

The model suffered from an inefficiency in the representation used by the solver: the search variable was `box_id`, that is the id of the box placed at time `t` (the array is indexed by `TIME`). This was extremely inefficient since collision checking constraints required verifying the absence of collisions between two boxes `b1` and `b2`, but the arrays containing the bounding boxes were indexed by `BOXES`, not `TIME`. In other words, to retrieve the AABB for the box `b1` placed at time `t`, one would have to write the following statement:

`box_bbox[box_id[t]]`

Code Snippet 5 shows how collision constraints are filled with this pattern, causing performance issues.

```
predicate no_boxes_above(var TIMESTEPS: t, var GRID_SIZE_Y: start_y) =
  let {
    % ...
  } in
  forall(t2 in 1..t) (
    % y collides
    -> (
      box_bbox_max_x[box_id[t2]] < box_bbox_min_x[box_id[t]]
      \/
      box_bbox_min_x[box_id[t2]] > check_end_x
    )
  )
;
```

Listing 5: `no_boxes_above` constraint using the `action_id` variable.

2.3 Intermediary modeling

The next step was to allow the gripper to pick up more than one box at a time. Implementing this directly in the Minizinc program would cause an exponential increase in the search space. We opted for the idea of the computation in two parts (as suggested by the assignment track), where:

1. the first program collapses sets of adjacent boxes into single actions that allow the gripper to pick up / drop the boxes in one move;

2. the array of actions is then fed to the Minizinc CSP that schedules the actions by checking the collision constraints.

The first model is written in an imperative way (C) and effectively boils down to clustering together adjacent boxes. This is done by first sorting the boxes and then checking for adjacency between each pair $(b_i, b_{i-1}) \mid \forall i \in \texttt{BOXES}$. Adjacent boxes are grouped into an action, which is represented by a unique id and a bounding box that precisely contains all involved boxes. The array of actions is a parameter of the Minizinc program that uses the new actions' AABBs for checking collisions.

```
for box in BOXES {
    if (Cluster c = same_cluster(box, previous_box))
        add_to_cluster(c, box);
    else
        new_cluster(box);
}
// compute the bbox (and its orientation) for each cluster
generate_actions(clusters);
```

Listing 6: clustering boxes into actions.

An action is effectively a pair: $(orientation, AABB)$, where the orientation is the same as that of the boxes (they all must have the same orientation) and the AABB is such that it precisely contains the boxes. Actions are then treated by the CSP as AABBs, exactly like single boxes were implemented before. The only difference is that actions are not all necessarily of the same size.

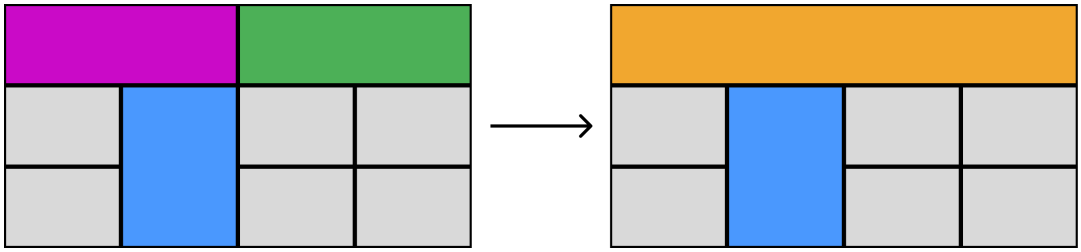


Figure 2: Grouping adjacent boxes (with same rotation) into actions. The left side shows 3 boxes, while the right side shows just 2 actions since the top boxes can be picked up together.

The Minizinc code had no significant changes since the collision constraints were operating on bounding boxes even before this current model.

2.3.1 A first attempt at symmetry breaking

Many solutions found with this model (and also the previous one) were redundant, that is symmetries of already-found solutions. This is because no order was imposed on the actions performed by the gripper. For example a row of three boxes could either be filled starting from the left or from the right. The number of symmetries was particularly high in the case of no `gripper overflow`, where less collision-related constraints were enforced.

To eliminate redundant solutions we tried to implement some flexible, non-strict ordering rules.

Actions were split into two groups: horizontal and vertical ones. Each group was then sorted by the following criterion:

$$greater_than(a_1, a_2) = \begin{cases} y_1 * pallet_width + x_1 - y_2 * pallet_width + x_2 > 0 & \text{if } r_1 = H \\ x_1 * pallet_height + y_1 - x_2 * pallet_height + y_2 > 0 & \text{if } r_1 = V \end{cases} \quad (1)$$

This criterion, however, was removing solutions. A counterexample is shown in Figure 3. With that pallet configuration (3 boxes of size 3×1) and a gripper size of 4, the green box (number 3) would have to be placed first, which is possible (since the ordering is enforced only within the class), but that would force the blue box (number 2) to be placed on the next timestep, which is impossible since $b_2 > b_1$ and number 1 still hasn't been placed. [1]

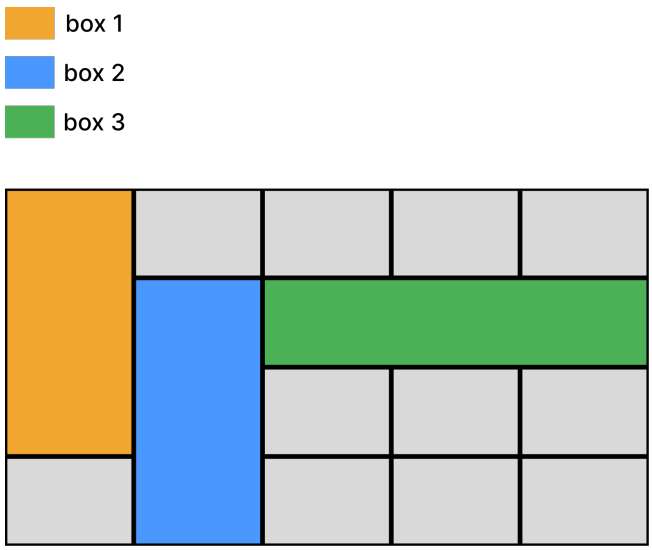


Figure 3: Counterexample for the symmetry breaking constraint.

2.4 Changing representation

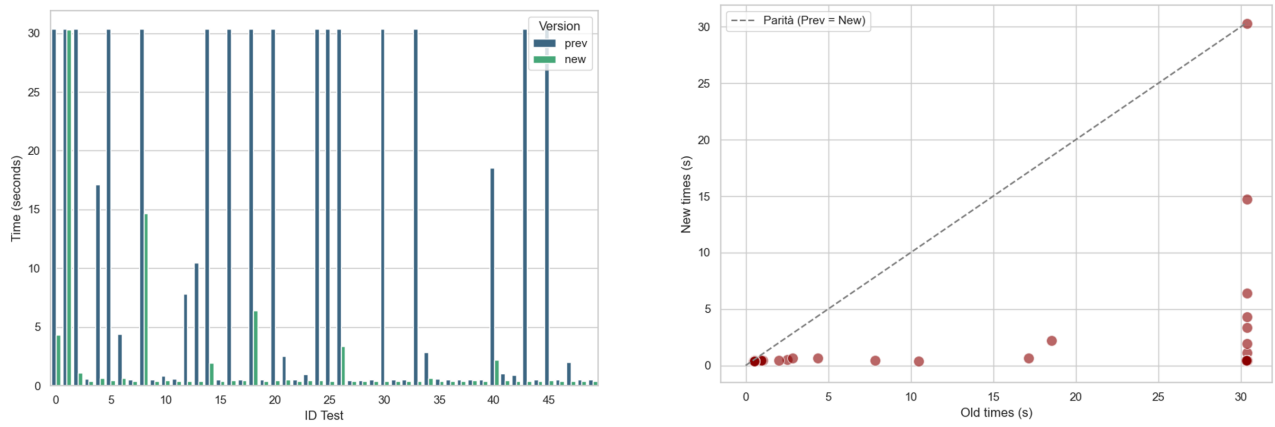
The 4th model aims to improve the performance of the previous that suffered from a bad choice in terms of representation (see Section 2.2). We changed the representation from using `action_id` (action taken at time `t`) to `action_time`, that is the time `t` when the action `i` takes place.

The decisional variable then becomes:

```
array[ACTIONS] of var TIME: action_time;
```

The collision constraints can thus be rewritten like shown in the following snippet.

[1] the identifiers generated by the current C code would actually be different (the constraint would obviously still be removing solutions), with green having $id = 1$ and orange $id = 2$ since horizontal boxes are assigned identifiers first.



(a) Times of old vs new representation

(b) Speedup

Figure 4: Performance of new and old representation (30 sec timeout)

```
predicate no_boxes_above(var ACTIONS: current_action, var TIME: t, var
    GRID_SIZE_Y: start_y) =
  let {
    var GRID_SIZE_X: check_end_x = actions_bbox_max_x[current_action];
    var GRID_SIZE_Y: check_end_y = actions_bbox_min_y[current_action];
  } in
  forall(other_action in ACTIONS) (
    (action_time[other_action] < t
      /\ actions_bbox_max_y[other_action] >= start_y
      /\ actions_bbox_max_y[other_action] < check_end_y) ->
    (
      actions_bbox_max_x[other_action] < actions_bbox_min_x[current_action]
      \/
      actions_bbox_min_x[other_action] > check_end_x
    )
  )
;
```

Listing 7: `no_boxes_above` collision constraint with the `action_time` representation.

The current model only needs this representation, but the CSP that we will introduce in Section 2.5 will need both, hence we'll have to introduce constraints to link the two representations together (they are very close to being the inverse of one another). This simple change provided a significant improvement in execution times as shown in 4.

2.5 Conveyor belt mechanism

The next step was to implement the conveyor belt mechanism. The belt mechanism brings the boxes to a (finite) retrieving area where the gripper can pick them up. This forces the gripper to "take breaks" during certain times when not enough boxes are available to perform the correct action. Implementing the conveyor belt meant:

I allowing for `NULL-moves`;

II implementing the queue for both the boxes in the retrieving area and for the remaining area.

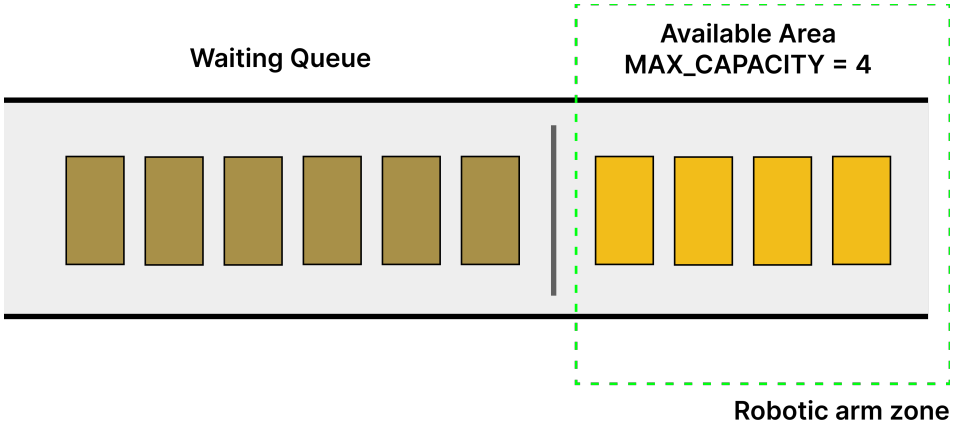


Figure 5: Conveyor belt being split into two areas: the waiting queue and the available queue.

2.5.1 Implementing `NULL-moves`

We implemented `NULL-moves` by using a support array of boolean type that indicates whether the action executed at time `t` is `NULL` or not:

```
array[TIME] of var bool:  active_move;
```

Two additional constraints were required to maintain consistency between the search variable `action_time` and the newly added `active_move`.

```
% link the two models together
constraint forall(action in ACTIONS) (
  action_id[ action_time[action] ] = action
);
% constraint inverse(action_time, action_id);
constraint forall(t in TIME) (
  active_move[t] == false -> action_id[t] = 0
);
constraint forall(t in TIME) (
  active_move[t] == true -> action_id[t] != 0
);
```

Listing 8: Constraints required to enforce consistency between the two decisional variables `action_time` and `active_move`.

The last two constraints ensure that whenever a padding move happens at time `t`, the corresponding action performed at `t` is a placeholder for the `NULL-move`. The action performed at time `t` is extracted by the previously mentioned `action_id` (see Section 2.2). This meant that the two representations had to be linked together via constraint. The first constraint in Code Snippet 8 does exactly that. [2]

2.5.2 Implementing the Conveyor Belt

The retrieving area (also called `available_queue` in the CSP) is a queue with a maximum capacity defined by an input parameter `N_MAX_AVAILABLE_BOXES`. If the area is not full at time `t`, then one box flows in during `t` (effectively moving from the waiting area to the retrieving one).

The constraints used to implement this logic are presented in Code Snippet 9.

```
% update number of boxes in available queue
constraint forall(t in 2..MAX_TIME) (
  let {
    var BOXES: requested_boxes = actions_num_boxes[action_id[t-1]];
  } in
  n_available_boxes[t] =
    n_available_boxes[t-1] - requested_boxes + min(n_boxes_waiting[t-1], 1)
);
% decrease number of boxes in waiting queue
constraint forall(t in 2..MAX_TIME) (
  n_boxes_waiting[t] = max(n_boxes_waiting[t-1] - 1, 0)
);
```

Listing 9: Constraints implementing the conveyor belt mechanism.

[2] Note that the global constraint `inverse` is not enough in this case since not all the values in each array are within the index set of the other array (0 being a value for `action_time` but not an index for `action_id`).

The number of boxes at time `t` is equal to:

$$\Delta_{boxes}(t) = \begin{cases} 1 & n_boxes_waiting(t) > 0 \\ 0 & otherwise \end{cases}$$

$$n_available(t) = \begin{cases} n_boxes(t-1) - n_requested_boxes(t-1) + \Delta_{boxes} & \forall t \in 2..\texttt{MAX_TIME} \\ \texttt{N_MAX_AVAILABLE_BOXES} & t = 1 \end{cases} \tag{2}$$

2.5.3 Padding Moves Symmetries

Allowing NULL-moves means new symmetries since those NULL-moves can be "placed" whenever the solver wants. Hence, we've added a constraint that aims to remove those newly-introduced symmetries regarding padding moves:

```
predicate is_executable(var ACTIONS: action, var TIME: t) =
  n_available_boxes[t] >= actions_num_boxes[action]
;

constraint forall(t in TIME) (
  (active_move[t] == false) ->
  forall(a in ACTIONS) (
    if (action_time[a] > t) then
      not is_executable(a, t)
    endif
  )
);
```

Listing 10: Constraint for removing NULL-moves symmetries.

This simply states that padding moves must be executed if and only if no action is executable at that time `t`. An action is executable if the number of boxes in the available queue is greater or equal than the number of boxes contained within that action.

3 Extra

3.1 Global constraint

Minizinc provides global constraints to ensure no overlap between bounding boxes, mainly `diffn`. This constraint has 4 array parameters that define the AABBs to check collisions on (origin point in 2D plus width and height). These constraints can replace our collision constraints (e.g.

`no_boxes_above`) by making some adjustments. The "default" `no_boxes_above` constraint ensures the absence of boxes within a given range (usually the gripper opening, which is 1). This can also be viewed as the following procedure:

1. extend the current action AABB by some amount (e.g. the gripper opening) in a certain direction;
2. ensure no collisions between the just extended AABB and all other, already placed, bounding boxes.

The `no_boxes_above` constraint becomes the following:

```
predicate no_boxes_above_global(var ACTIONS: current_action,
                                var TIME: t,
                                var GRIPPER_OVERFLOW: extend_by) =
  let {
    var GRID_SIZE_X: origin_x = actions_bbox_min_x[current_action];
    var GRID_SIZE_Y: origin_y =
        max(1, actions_bbox_min_y[current_action] - extend_by);
    var GRID_SIZE_X: size_x = actions_width[current_action];

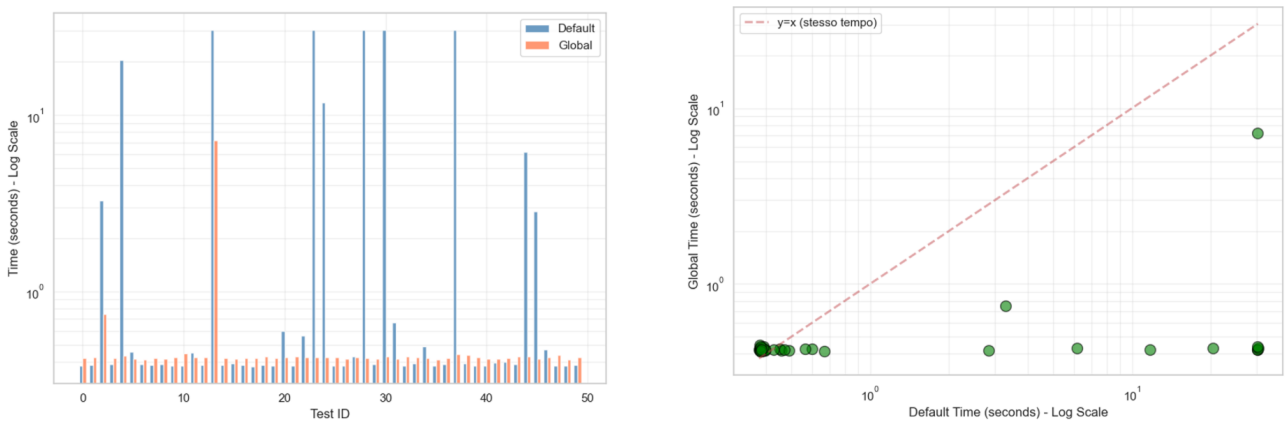
    var GRID_SIZE_Y: max_y = actions_bbox_max_y[current_action];
    var GRID_SIZE_Y: size_y = max_y - origin_y + 1;
  } in
  forall(other_action in ACTIONS) (
    (action_time[other_action] < t) ->
      diffn([origin_x, actions_bbox_min_x[other_action]],
            [origin_y, actions_bbox_min_y[other_action]],
            [size_x, actions_width[other_action]],
            [size_y, actions_height[other_action]])
  )
;
```

Listing 11: Constraint for removing NULL-moves symmetries.

3.2 Symmetries

The last presented model suffers from symmetries that make proving unsatisfiability of the CSP really expensive in terms of time. Symmetries related to the padding moves introduced by the conveyor belt logic have been removed by only allowing NULL-moves if strictly necessary.

However, the symmetries that stem from all the possible orders of actions performed by the robotic arm to reach the goal pallet configuration have not been completely removed. The following constraint makes the solver "prioritize" the `TOP` and `RIGHT` rotations (the other two are used only when necessary).



(a) Execution time per test (b) Speedup

Figure 6: Default vs global constraint (log scale)

```
constraint forall(action in ACTIONS) (
  % left allowed iff there's a box to the right
  (action_rotation[action] == LEFT -> not no_boxes_right(...)
  /\
  % bottom allowed iff there's a box above
  (action_rotation[action] == BOT  -> not no_boxes_above(...)
);
```

Listing 12: Constraint for removing NULL-moves symmetries.

This doesn't eliminate all symmetries related to rotations since the solver is capable of performing the actions in an order that allows opening the gripper to the left (or down).

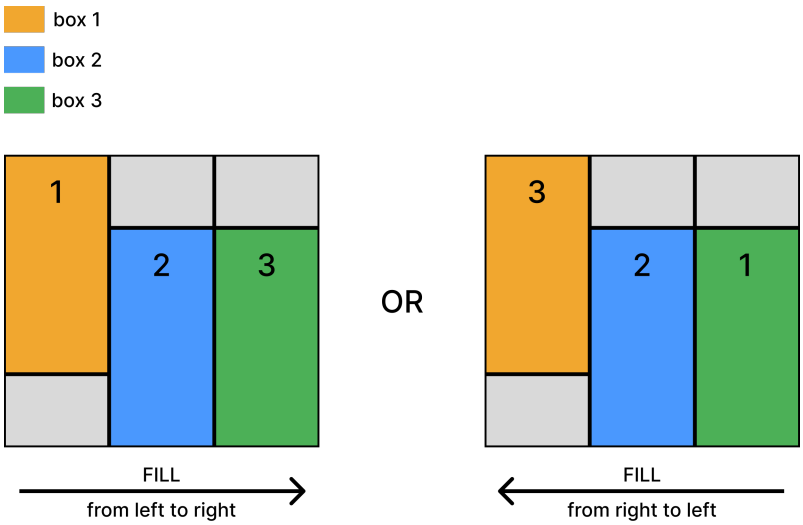


Figure 7: Symmetries caused by action ordering and rotations.

After numerous attempts we couldn't fully solve this issue because enforcing an order on actions can remove real solutions (not just symmetries, see Section 2.3.1). Another approach we tried is to impose an order on clusters of boxes that are not interfering with each other. The ordering of the clusters would have to be decided beforehand, that is before running the Minizinc model. This however has three main problems:

- the average pallet doesn't have clearly separated clusters. Pallet configurations usually aim to tightly pack boxes together in order to avoid wasting space;
- it is not complete, meaning it will never remove all symmetries (always an approximation);
- because this is done before the solver finds a solution, it's impossible to know where the gripper will open (given a specific action on the pallet), hence the clustering has to account for all possible openings, approximating even further the clustering results.

3.3 Generating Redundant Constraints

In the case of gripper overflow (i.e. the gripper is wider than the box it is picking up) there's a pattern (shown in Figure 3.3) that enforces order on the involved boxes. This order is eventually found by the solver, but it can also be easily found by scanning the pallet during the first step that generates the Minizinc input (see Section 2.3). This allows to generate additional (and redundant) constraints that can help the solver in its search.

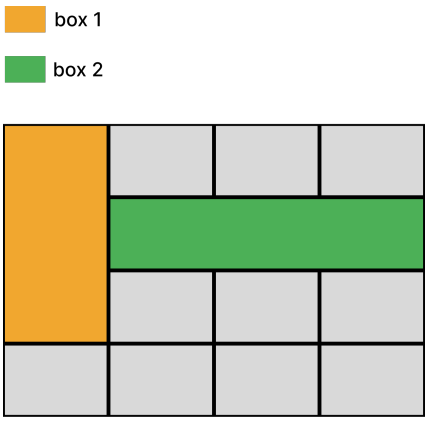


Figure 8: "T" like pattern that forces an order on the actions.

In the case of Figure 3.3, box number 2 has two be placed before box number 1. At that point we can generate a constraint like the following:

```
constraint action_time[a1] < action_time[a2];
```

This could be extended to generate constraints that can be derived by transitivity (same example as the one in Figure 3.3 but with another vertical box to the left of box number 1).

Configuration (Box_Pallet)	**SAT Instance**	**Timeout (300s)**	**AVG Time (s)**
b20x10_p500x500	5/5	0	0.36
b20x10_p900x500	3/3	0	4.16
b20x10_p900x700	1/2	1	246.38
b20x10_p900x900	1/2	1	229.52
b30x10_p700x700	4/4	0	0.40
b40x20_p1100x1100	5/5	0	0.38

Table 1: With redundant constraints

3.4 Testing Automation

Ideally we would like to be able to randomly generate pallet configurations and feed them to our pipeline and assert that the return value is equal to the one we expect. However, with the pallets being randomly generated we need a way of computing an expected value. It turns out we can approximate this return value because some pallet configurations can be proved to be unsatisfiable even without running the entire CSP, but just by doing checks on a graph.

This method is not complete, meaning that it doesn't capture all cases of unsatisfiability, but if `UNSAT` is returned by the first check, then we know for sure that the CSP has to be `UNSAT` too.

First of all, a pallet configuration can only be impossible to achieve if there is gripper overflow (i.e. the gripper size is greater than the boxes' width). We implement two different checks: one for loops and another one for a specific pattern that immediately results in unsatisfiability of the CSP.

3.4.1 Loops

Gripper overflow makes it so that a loop pattern like the one shown in Figure 3.4.1 causes the pallet configuration to be unsatisfiable. The reason is that box 3 (red) has to be placed before box 2 (green) (by transitivity), but box 2 (green) must be placed before box 3 (red) since.

We can determine whether a box `b1` has to be placed before another box `b2` if `cuts_through(b1, b2)` holds. The function `cuts_through(b1, b2)` can be defined as:

```
static u8 cuts_through(const Action* a1,
                                const Action* a2)
{
  const u32 x_overflow = (a1->rotation == V) ?
                            0 : a1->gripper_overflow;
  const u32 y_overflow = (a1->rotation == H) ?
                            0 : a1->gripper_overflow;

  const AABB a1_extended_bbox =
    AABB_extend(a1, x_overflow, y_overflow);
  return collide(a1_extended_bbox, a2->bbox);
}
```

Listing 13: clustering boxes in to actions.

Using this function we extract a graph where the directed edge `(a1, a2)` exists if and only if `cuts_through(a1, a2)`. We then check for loops on that graph. The presence of a loop implies the unsatisfiability of the CSP.

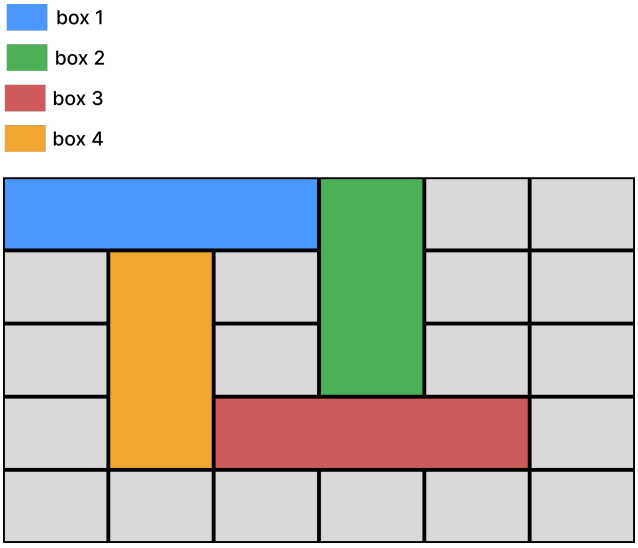


Figure 9: `UNSAT` loop pattern with gripper overflow.

3.4.2 Double "T" Pattern

The second pattern that guarantees unsatisfiability is shown in Figure 3.4.2. This pattern can be found using the graph that is built during the previous check (see Section 3.4.1). We look for a node `x` that has two incoming edges originating from nodes that lie on both sides of node `x` (one to the left and another to the right).

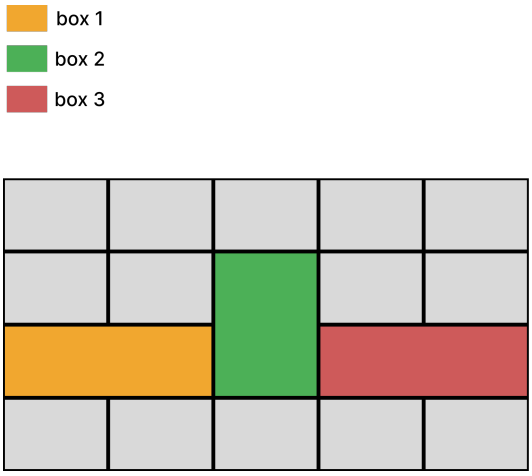


Figure 10: `UNSAT` "double t" pattern.

4 Results

Table 2 shows results obtained by running the model with input configurations that were generated by OnPallet (2026). All tests were run using a gripper with width equal to that of the boxes to force satisfiability.

Configuration	**Without Global Constraints**			**With Global Constraints**		
	SAT	**Timeout**	**AVG (s)**	**SAT**	**Timeout**	**AVG (s)**
b200x100_p500x500	5/5	0	0.36	5/5	0	0.36
b200x100_p900x500	3/3	0	4.80	3/3	0	4.17
b200x100_p900x700	1/2	1	288.00	1/2	1	246.14
b200x100_p900x900	0/2	2	300.28	1/2	1	229.68
b300x100_p700x700	4/4	0	0.39	4/4	0	0.40
b400x200_p1100x1100	5/5	0	0.37	5/5	0	0.37

Table 2: Performance comparison with and without global constraints

The next table shows results obtained by running the model with input configurations that were randomly generated. All tests were run using a gripper width equal to that of the boxes to force satisfiability.

5 Conclusions

List of figures

1 Example of final result 2
2 Grouping adjacent boxes (with same rotation) into actions. The left side shows 3 boxes, while the right side shows just 2 actions since the top boxes can be picked up together. 6
3 Counterexample for the symmetry breaking constraint. 7
4 Performance of new and old representation (30 sec timeout) 8
5 Conveyor belt being split into two areas: the waiting queue and the available queue. . 9
6 Default vs global constraint (log scale) 13
7 Symmetries caused by action ordering and rotations. 14
8 "T" like pattern that forces an order on the actions. 15
9 `UNSAT` loop pattern with gripper overflow. 17
10 `UNSAT` "double t" pattern. 17

List of tables

1 With redundant constraints 15
2 Performance comparison with and without global constraints 18

References

OnPallet (2026). Onpallet - pallet loading software. https://www.onpallet.com/index.php. Accessed: 2026-01-19.